

WanderMatch

Social & Group Travel Planning,
with an AI Consensus Planner

TEAM- Chaosminds

#	Member Name	Role
1	Nikitha M N	Frontend / Real-time UI
2	Panchami P	Backend / PostgreSQL & Voting Logic
3	Sharvani G Bhaskar	AI Consensus Planner
4	P Nithya	Auth / Presence / Deployment

2. Problem Understanding

The problem behind the product

Group trip planning breaks down when one or two people disagree, nobody wants to block the group, and the disagreement is either steamrolled or lost in group-chat back-and-forth.

Existing shared-itinerary tools solve collaborative editing, but not the harder question: when preferences conflict, who decides, how, and on what basis?

WanderMatch combines a live shared itinerary, structured proposal-and-vote flow, and an AI layer that reads the reasons behind objections — not just the yes/no tally — to produce common ground or, when preferences are genuinely incompatible, parallel branches.

The design goal is a system that never silently overrides anyone, never gets permanently stuck, and never treats an LLM suggestion as authoritative: the admin or a unanimous group agreement can always override it.

3. Scope

What is built — and what is deliberately left out

BUILD: trip creation/invites/auth; live PostgreSQL itinerary with FastAPI WebSockets; proposal + Yes/No/Abstain voting; Mode A Admin-Led consensus; Mode NA No-Admin common-ground + branching; recursive multi-way branches; mandatory Trip Chat; solo-to-group and solo-to-guide matching; optional face registration; Cloudinary trip photos + DeepFace tagging.

LEFT OUT: admin-inactivity fallback; live self-hosted LLM serving; live face recognition during judging (fixed demo photos are pre-processed); AI follow-up questioning; live translation of user-generated content; deep AI-training detail.

4. User Journey

How a traveller moves through WanderMatch

Sign-up → Firebase Auth → age, languages and travel interests → optional three-photo face registration → create/join a trip and choose Admin-Led or No-Admin mode.

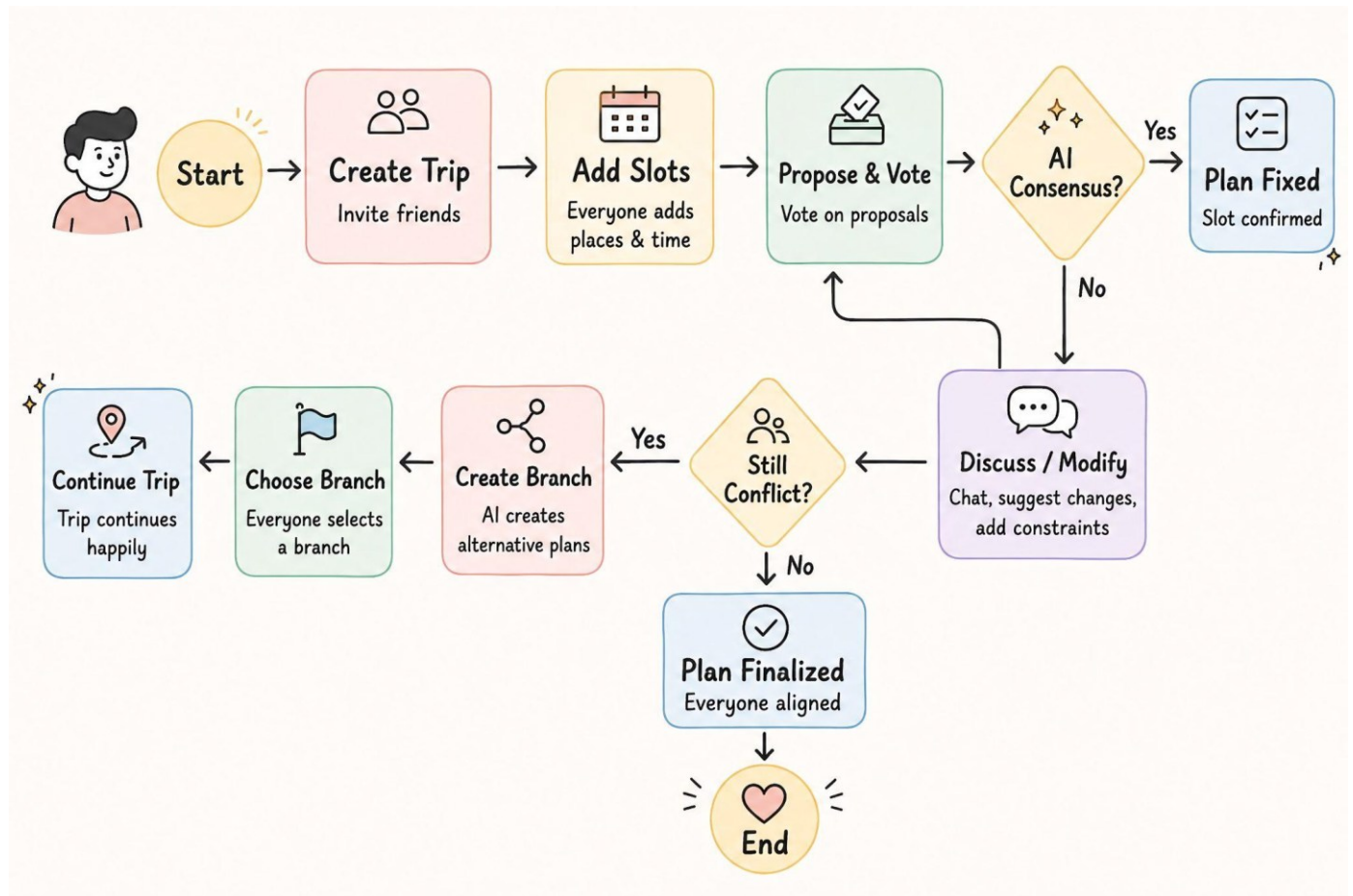
Trip Home shows day-by-day slots with statuses: Empty / Proposing / Voting / Common-Ground Round n / Branching / Confirmed.

Slot Detail exposes every proposal, live tally and every stated No-reason. Members submit proposals, vote Yes, or vote No with a mandatory typed reason.

If disagreement triggers AI, the AI candidate appears alongside the original. If branching occurs, members see their branch preview and can Confirm or Request Modification.

Solo travellers get ranked group matches with reasons, or guide matches using language, specialisation, price and rating. Trip Photos offers shared gallery + optional personal face-tagged albums.

User Journey Diagram:



5. Architecture

Simple single-orchestrator design

Layer	Responsibility
React + TypeScript	Trip Home, Slot Detail, Branch View, Chat, Solo Match, Face Registration
FastAPI	Single orchestrator; REST endpoints; per-trip WebSocket broadcast manager
PostgreSQL	PS-11 schema + additive tables; system of record
Hosted LLM API	Consensus generation and branch synthesis
Cloudinary	Photo storage, thumbnails and delivery
Face service	Python + DeepFace / ArcFace
Deployment	Vercel/Render + managed PostgreSQL

React never talks directly to PostgreSQL, the LLM API, Cloudinary or the face service. Every request and state-changing broadcast passes through FastAPI. WebSockets provide one live channel per trip. A Redis-backed connection manager can be swapped in later for multi-instance scaling.

5A. Consensus & Conflict Resolution — Detailed Flow

The timer only exists once there's a No. There is no fixed voting window starting at proposal creation. The 10-minute response window starts when the FIRST No lands on a slot.

Mode A — Admin-Led

#	Step	Behaviour
1	Admin creates base itinerary.	Members immediately see it and can propose alternatives.
2	Member submits proposal.	Proposal appears live to everyone.
3	Members vote.	No requires a typed reason in votes.comment; every reason is visible.
4	Admin invokes AI after a No.	AI recommendation panel shows every holdout suggestion.
5	AI attempts common ground.	It blends compatible suggestions before branching.
6	Admin presents candidate.	Admin may repeat, accept, reject, override or branch.
7	Trip Chat override.	Unanimous agreement can terminate the AI process immediately.

Mode NA — No-Admin

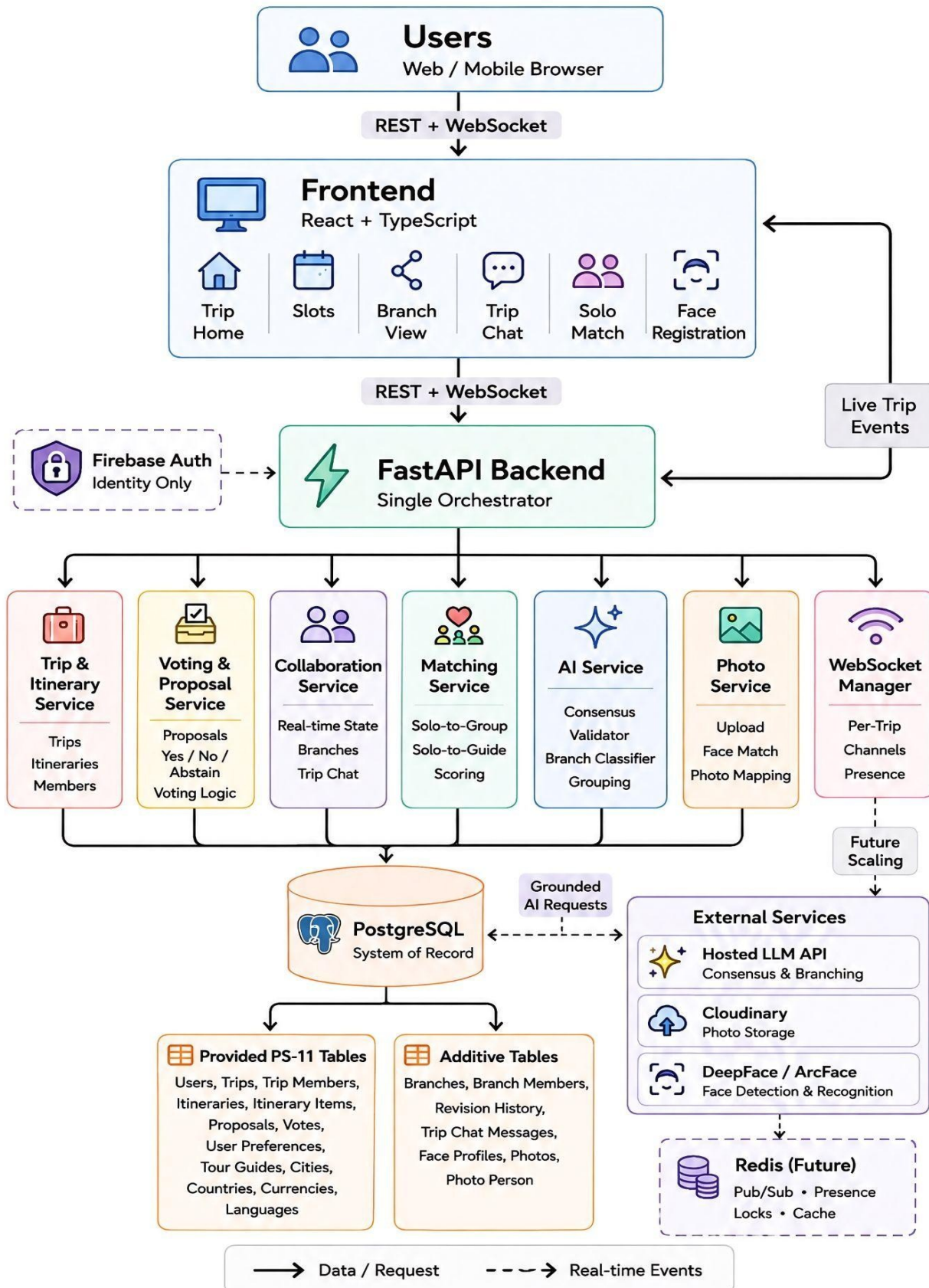
#	Step	Behaviour
1	Proposal added.	No timer starts.
2	Members vote.	No requires the member's own typed suggestion.
3	No No exists.	Plan carries forward; no repeating background timer.
4	FIRST No arrives.	10-minute response window begins.
5	AI creates blended plan.	One candidate; original remains visible.

#	Step	Behaviour
6	Group re-votes.	Unanimous Yes confirms; another No triggers another common-ground round.
7	Fixed/incompatible requirement.	Switch to branching when common ground is exhausted.
8	Multi-way branching.	One branch per genuinely distinct incompatible preference; near-duplicates merge.
9	Recursive branching.	Any internally divided branch repeats the same process to any depth.
10	Branch preview.	Confirm / Request Modification; silence = accepted after window.
11	Revision cap.	Soft-capped revision cycle finalises the branch after the cap.
12	Activity ends.	Branches apply only to that activity/time window; next slot starts unbranched.

Edge cases

Scenario	Resolution
No without reason	Blocked at UI/API level.
No one ever votes No	No timer; plan carries forward.
New proposal during active round	Queued for next decision cycle.
Multiple Yes votes by same member	Only one active Yes; prior Yes is retracted.
Member silent during round	Non-no; may still object later.
3rd unresolved common-ground round (NA)	AI evaluates fixed requirements and branches automatically if exhausted.
3rd unresolved round (Admin)	AI recommends branching; admin decides.
3+ genuinely distinct holdouts	3+ branches, not binary.
Single-person branch	Auto-resolves.
Branch disagrees internally	Recursive consensus/branching.
Unanimous Trip Chat agreement mid-round	Immediately overrides/terminates current AI process.
Low-confidence face match	Unknown; never force-tagged.

Architecture Diagram:



6. Data Model Usage

Use the provided schema; extend additively

Provided table	Use
trips	Trip dates, destination, party, budget; group-trip flag.
trip_members	Membership + role; owner is Mode-A admin.
itineraries	Versioned plan; version used for optimistic concurrency; generated_by records source.
itinerary_items	One row per slot/activity; status + locked state.
proposals	add/remove/replace/reschedule actions; closes_at holds the 10-minute window; status drives slot state.
votes	yes/no/abstain; comment is mandatory for No.
users	Traveller identity; locale.
user_preferences	Interests, languages, pace, budget for compatibility.
tour_guides	Language, specialisation, price, rating for guide matching.
cities/countries/currencies/languages	Reference data and multilingual metadata.
New table	Purpose
branches	Recursive branch tree via parent_branch_id.
branch_members	Member-to-branch assignments.
revision_history	Every AI-generated candidate revision.
trip_chat_messages	Mandatory always-on Trip Chat.
face_profiles	Three reference embeddings per registered member.
photos / photo_person	Cloudinary photo records and face tags.

Conventions: opaque prefixed IDs; decimal + ISO-4217 money; ISO-8601 timestamps; exact lowercase snake_case enums; BCP-47 language tags. The provided PS-11 dataset was checked with starter queries and conformance tooling before finalising this design.

7. AI Approach

No AI feature asks a member follow-up questions. A No vote carries one direct member-typed suggestion in `votes.comment`; AI reads that text as submitted.

Feature	Purpose	Verification
Blended common-ground generator	Produces one candidate accommodating as many active suggestions as possible before branching.	Structured JSON + hard-constraint validator; scripted disagreement tests.
Branch-trigger classifier	Classifies whether a requirement is still accommodable or genuinely fixed.	Labelled keep-blending vs branch scenarios.
Multi-way branch grouping	Creates as many distinct branches as required; merges near-duplicates; supports recursion.	3–4-way and overlapping-preference scenarios.
Solo-to-group matcher	Explainable weighted score over age group, languages, interests and dates.	Fixed profiles + expected top matches.
Solo-to-guide matcher	Scores language, specialisation and price.	Fixed profile test set.
DeepFace / ArcFace	Face detection, embeddings and member matching from registered references.	Confidence threshold; low-confidence remains Unknown; fixed demo batch.

8. Tech Stack

Component	Why
React + TypeScript	Fast, type-safe UI and state-machine implementation.
Firebase Auth	Low-friction identity; UID mapped to <code>users.user_id</code> .
PostgreSQL	Direct fit for provided relational dataset and voting/proposal model.
FastAPI + WebSockets	Single backend orchestrator and live trip state.
Hosted LLM API	Reliable sprint/demo inference.
Cloudinary	Managed photo storage and delivery.
DeepFace (ArcFace)	Practical face detection/recognition wrapper.
Vercel / Render + managed Postgres	Fast team-operable deployment.

9. 24-Hour Execution Plan

REPLACED WITH THE TEAM'S LATEST EXECUTION PLAN

Team: 4 people · Start: Day 1, 12:00 (noon) · End: Day 2, 12:00 (noon). Hard constraint: all feature work stops at 09:00 Day 2 (Hour 21); nothing new after that, stabilization and rehearsal only.

Roles

Person	Track	Owns
Nikitha M N	Frontend	React + TypeScript UI — Trip Home, Slot Detail, Branch View, Chat, Solo-Match, Face Registration screens
Panchami P	Backend	FastAPI, PostgreSQL (provided PS-11 schema + additive tables), REST endpoints, WebSocket broadcast manager, voting/proposal logic
Sharvani G Bhaskar	AI	Blended common-ground generator, hard-constraint validator, branch-trigger classifier, multi-way branch grouping, compatibility scoring logic
P Nithya	Setup & Integration	Repo/branch setup, PostgreSQL provisioning + seeding, Firebase Auth project, Cloudinary, env/config; then floats into Backend → Auth/Face/Photos/Deploy

P4's setup is front-loaded into Hour 0–1 so nobody else is blocked past Hour 1.

Git / Branch Structure

Branch	Rule
main	Protected; only receives merges at checkpoints; always demo-safe.
develop	Shared integration branch; everyone merges here first; tested at each checkpoint.
feature/frontend, feature/backend, feature/ai, feature/setup	One long-lived branch per person, branched from develop.

- Pull develop before every new work block.
- Commit small and often.
- Merge into develop via PR; one teammate skims before merge.
- develop → main only after Checkpoint 1 and Checkpoint 2.
- No one touches main directly.

Module ownership

Area	Owner	Path
Frontend components/screens	Nikitha M N	/frontend/src/*
FastAPI routes, DB models, WebSocket manager	Panchami P	/backend/app/*
AI prompts, validators, classifiers	Sharvani G Bhaskar	/backend/ai/*
Config, env, Docker/deploy, Firebase/Cloudinary, face service	P Nithya	/infra/*, .env.example, /backend/services/face*, /backend/services/auth*

If two tracks need the same file, one owner is assigned for the day and changes are routed through that owner.

Hour-by-Hour Plan

Elapsed / Clock	Nikitha M N— Frontend	Panchami P— Backend	Sharvani G Bhaskar — AI	P Nithya— Setup & Integration
Hr 0–1 / 12:00–13:00	Sketch Trip Home + Slot Detail; agree one demo flow.	Load PS-11.db; starter_queries.sql; confirm additive tables.	Draft common-ground + branch-trigger prompts against real dataset examples.	GitHub branches; PostgreSQL provisioned/seeded; Firebase Auth; Cloudinary keys; .env.example.
Hr 1–4 / 13:00–16:00	Static Trip Home + Slot Detail UI.	FastAPI scaffold; CRUD trips/trip_members/proposals/votes.	Hard-constraint validator stub.	Join P2: in-memory per-trip WebSocket manager; test two clients.
Hr 4–8 / 16:00–20:00	Wire proposal/vote UI; live tallies.	Vote → WebSocket broadcast; itinerary optimistic-concurrency bump.	First blended-plan generation end-to-end on scripted disagreement.	Cloudinary upload endpoint + gallery scaffold.
Hr 8–12 / 20:00–00:00	Admin recommendation panel; AI candidate beside original.	closes_at 10-minute window; round counter.	Revision loop for 2–3 rounds.	3-photo face registration capture flow; standalone demo.
CHECKPOINT 1 / 00:00 / Hr 12	Everyone stops. Core working demo.	See below.	See below.	See below.
Hr 12–16 / 00:00–04:00	Branch View + Confirm/Request Modification.	branches + branch_members; recursive endpoint.	Branch classifier + multi-way grouping on 3–4-way scenario.	Trip Chat over WebSocket; unanimous-Yes override.
Hr 16–19 / 04:00–07:00	Solo-match + guide screens.	Group + guide compatibility endpoint.	Support scoring edge cases; prep scripted disagreement.	Pre-process fixed demo photos through DeepFace; verify tags.
Hr 19–21 / 07:00–09:00	Full click-through; flag breakages, don't fix in isolation.	Fix cross-endpoint integration issues.	Fix prompt/validator integration issues.	Fix auth/broadcast/photo integration issues.
CHECKPOINT 2 / 09:00 / Hr 21	Everyone stops. Organiser hard freeze.	No new features after this point.	No new features after this point.	No new features after this point.

Checkpoints

Checkpoint 1 — Core Working Demo (00:00, Hour 12): the system works end-to-end without AI. Everyone stops development, merges all four feature branches into develop, resolves conflicts together, runs sign-up → create/join trip → itinerary → proposal → Yes/No with reason → live WebSocket update, then merges develop → main as the safety net.

Checkpoint 2 — AI Working Demo (09:00, Hour 21): complete system works end-to-end. Merge remaining work, run proposal → No with reason → AI blended plan → re-vote → scripted branch → branch preview → confirm. Fix only critical bugs/integration breaks, then merge develop → main. Feature development ends here.

Final Phase — Demo Preparation (Hour 21 → Hour 24)

Time	Focus	Concrete action
Hr 21–22 / 09:00–10:00	Feature freeze	Whole team locks scope; no new features.
Hr 22–23 / 10:00–11:00	Stabilize	Fix only the rehearsed demo path; clean shown UI; verify seeded trip/disagreement/photos.

Time	Focus	Concrete action
Hr 23–24 / 11:00–12:00	Rehearse	Assign presentation roles; finalize script; run demo twice on actual machine; record offline fallback.

Optional Work / Buffer

- Help another track before adding scope.
- Strengthen tests/edge cases already built, especially branching.
- Only if Checkpoint 2 is comfortably early: deepen face recognition; otherwise leave it out.
- Rule for all 24 hours: a stable fully working demo beats a feature-complete shaky one. Cut scope rather than miss Checkpoint 2.

10. Risks and Fallbacks

Risk	Fallback
AI creates logistically broken plan	Hard-constraint validation; regenerate once; then branch/admin review.
Hosted/self-hosted model reliability	Use hosted LLM during sprint/demo; self-hosting is future optimisation.
Live face recognition fails	Use pre-processed fixed demo photo set; live-demo registration remains available.
Recursive branching complexity	One generic recursive resolver; rehearse one 3–4-way case.
Venue Wi-Fi fails	Recorded offline full-demo fallback on presentation machine.

11. Language Approach — English Only

- Dataset language metadata remains available through languages, users.locale, city language and guide language fields.
- React UI is English-only using a key-based string table; no translation service is required.
- Language is still used in guide matching and solo/group compatibility.
- User-generated chat and vote-reason text is displayed as typed; no live translation is built.
- XR device declaration: not applicable — WanderMatch has no AR/VR component.

Appendix. Business Benefits

- Fewer stalled trips: every disagreement has a concrete resolution path.
- No silent overrides: every No and reason is visible and preserved.
- Faster consensus: AI compresses negotiation into a candidate proposal per round.
- Branching avoids forced compromise when preferences are genuinely incompatible.
- Solo travellers get a real on-ramp into compatible groups or guides.
- Trip memories require little manual sorting through opt-in face-based grouping.
- Always-on Trip Chat keeps discussion and decision-making in one place.